

Experiment No. 3: Perceptron with Fixed Increment Learning Algorithm (Binary Step Function)

1. Aim/Purpose

To implement a perceptron using a fixed increment learning algorithm with the binary step activation function and train it until no further changes in weights are required.

2. Theory

The perceptron is an artificial neural network model used for binary classification. The binary step function is a simple activation function that outputs either 0 or 1 depending on whether the weighted sum is above a threshold. The perceptron updates its weights iteratively based on classification errors until convergence. A perceptron learns by adjusting its weights based on misclassified examples using a fixed increment method. The learning process continues until the perceptron classifies all training data correctly or reaches a predefined iteration limit.

3. Algorithm:

1. Define the AND function as input-output pairs.
2. Initialize weights and bias to small random values.
3. For each training sample:
 - Compute the output using the weighted sum.
 - Update the weights using the rule: where η is the learning rate, t is the target output, and $\hat{y}$ is the predicted output.
 - Here the major difference is the application of the binary step inside the loop providing more modularity.
4. Repeat until convergence.
5. Test the trained perceptron on AND function inputs.

4. Implementation:

```
import numpy as np
import matplotlib.pyplot as plt

# Binary Step Activation Function
def binary_step(x):
    return 1 if x >= 0 else 0

# Training Data (AND Function)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])

# Initialize weights and bias
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
converged = False
iterations = 0
decision_boundaries = []

# Perceptron Training Loop
while not converged:
    iterations += 1
    previous_weights = weights.copy()
    previous_bias = bias

    # Training step
    for i in range(len(X)):
        weighted_sum = np.dot(X[i], weights) + bias
        prediction = binary_step(weighted_sum)
        error = y[i] - prediction

        # Update weights and bias
        weights += learning_rate * error * X[i]
        bias += learning_rate * error

    # Store decision boundary after each epoch
    if weights[1] != 0:
        x_vals = np.linspace(-0.5, 1.5, 100)
```

```

        y_vals = -(weights[0] * x_vals + bias) / weights[1]
        decision_boundaries.append((x_vals, y_vals))

    # Check for convergence
    if np.array_equal(previous_weights, weights) and previous_bias == bias:
        converged = True

# Plot decision boundary evolution
plt.figure(figsize=(6, 6))
for i, (x_vals, y_vals) in enumerate(decision_boundaries):
    plt.plot(x_vals, y_vals, label=f'Epoch {i+1}')

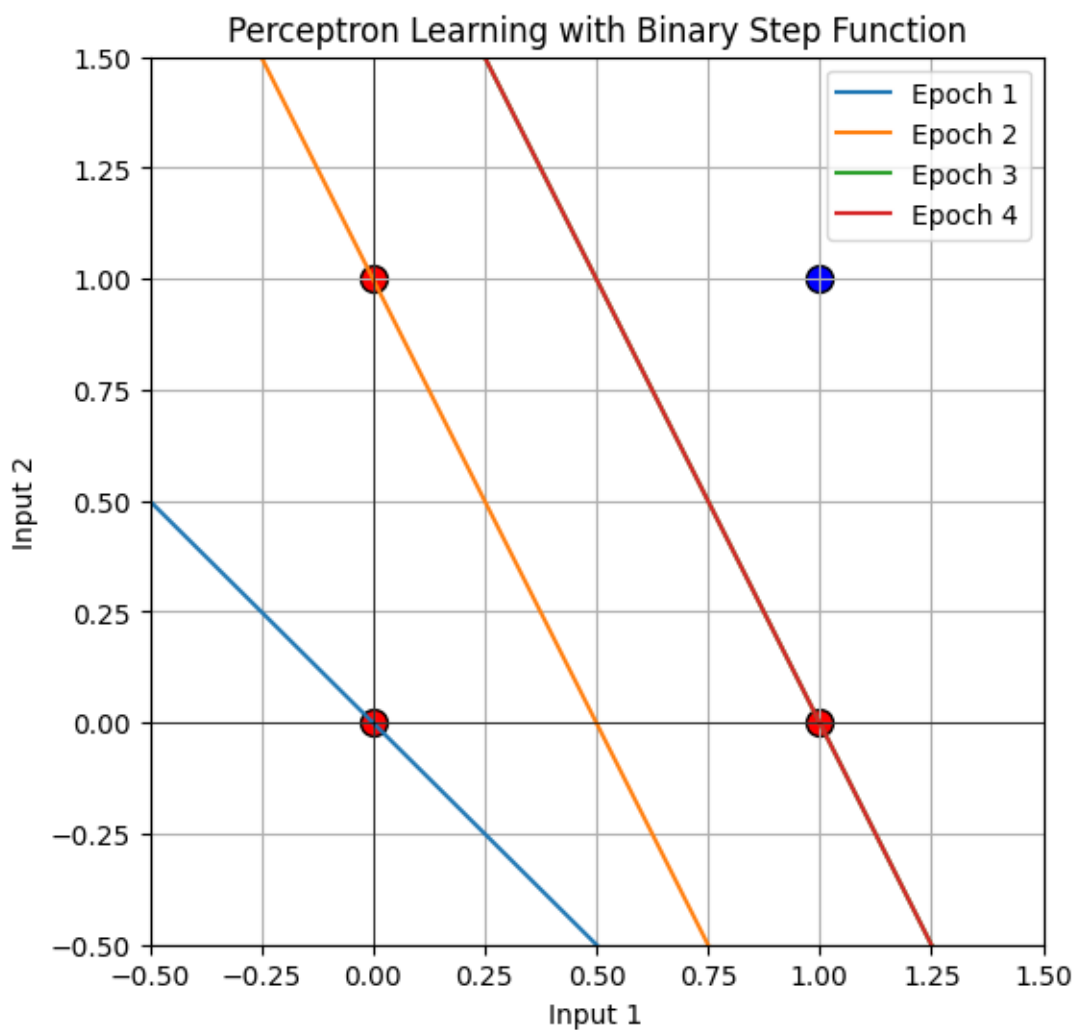
# Plot data points
for i in range(len(X)):
    plt.scatter(X[i][0], X[i][1], c='red' if y[i] == 0 else 'blue', marker='o',
                edgecolors='black', s=100)

plt.xlabel('Input 1')
plt.ylabel('Input 2')
plt.title('Perceptron Learning with Binary Step Function')
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.grid(True)
plt.show()

# Display final weights and number of iterations
print(f'Final Weights: {weights}')
print(f'Final Bias: {bias}')
print(f'Number of Iterations Until Convergence: {iterations}')

```

5. Output



Final Weights: $[0.2 \ 0.1]$

Final Bias: -0.20000000000000004

Number of Iterations Until Convergence: 4

6. Conclusion

The perceptron successfully classifies the AND function, demonstrating convergence within a specific number of iterations using binary step function.